

Idea: Reality MVP

One-liner:

Turn a rough product sketch + text prompt into a 3D model, then let users place it in the real world through AR.

Demo line:

“Codex helps builders ship faster. Reality MVP helps product teams go from sketch to spatial prototype before committing to a full build.”

Workflow

1. User uploads a sketchy product image
Example: rough drawing of a smart bottle, lamp, desk gadget, chair, etc.
2. User enters a text prompt
Example: “A smart water bottle for gym users that glows when hydration is low.”
3. OpenAI vision analyzes the sketch
It extracts product type, shape, features, materials, and intended use.
4. App creates a refined 3D-generation prompt
Example: “Create a small cylindrical smart hydration bottle with LED ring...”
5. 3D generation API creates the model
Use Meshy or similar to generate a .glb / 3D asset from text or image. Meshy officially supports Text-to-3D and Image-to-3D APIs. [oai_citation:0±Meshy Docs](https://docs.meshy.ai/api/text-to-3d?utm_source=chatgpt.com)
6. Codex generates the AR prototype app
Codex creates or updates the AR page, product config, feature callouts, README, AGENTS.md, and validation/build plan.
7. User scans QR code on phone
Opens a mobile web AR page.
8. User places the generated model in the real world
Use <model-viewer> for browser-based 3D/AR display. It supports AR modes including webxr, scene-viewer, and quick-look. [oai_citation:1±Model Viewer](https://modelviewer.dev/docs/?utm_source=chatgpt.com)

Why this fits the Codex theme

This should not be pitched as just “AI 3D model generation.”

The stronger pitch is:

“Codex turns a rough product concept into a runnable AR prototype application.”

OpenAI vision can analyze image inputs, but it does not directly generate complete AR apps. [oai_citation:0±A 3D API can generate the model, but it does not build the surrounding product

experience. Meshy supports text/image-to-3D generation, but that is only one piece of the workflow. [OBJ]

Codex is the bridge that creates the actual software layer:

- AR viewer page
- model config
- feature labels
- product landing page
- PWA/mobile route
- README
- AGENTS.md
- MVP_SPEC.md
- validation plan

Codex also supports project instructions through AGENTS.md; OpenAI's docs say Codex reads AGENTS.md files before doing work. [OBJ] OpenAI describes Codex as a coding agent that can write features, answer questions about a codebase, fix bugs, and propose pull requests. [OBJ]

Architecture

Frontend:

Next.js + TypeScript + Tailwind

Pages:

- / Upload sketch + enter prompt
- /result Show product analysis, generated 3D status, Codex build pack
- /ar/[id] Mobile AR viewer page
- /build-pack AGENTS.md, MVP spec, validation plan, generated tasks

Backend:

- /api/analyze-sketch
- /api/generate-3d
- /api/generate-build-pack
- /api/run-codex or local script wrapper

AI / services:

- OpenAI vision: analyze uploaded sketch
- Optional OpenAI image generation: create cleaner product concept art
- Meshy or similar: generate 3D model from text/image
- Codex: generate AR app files and build pack

AR:

- <model-viewer>
- .glb model output
- QR code to open AR page on phone

<model-viewer> is the safest AR choice because it is designed to display interactive 3D models on the web and in AR. [OBJ] Google's ARCore docs also describe <model-viewer> as

a path for viewing and placing 3D models in AR, with Scene Viewer fallback when WebXR is unavailable. [OBJ]

Technology stack

App:

- Next.js
- TypeScript
- Tailwind CSS

AI:

- OpenAI vision API for sketch analysis
- Optional OpenAI image generation for cleaned concept image
- Codex / codex exec for app generation and build-pack generation

3D:

- Meshy Text-to-3D or Image-to-3D API
- Backup placeholder .glb model library

AR:

- <model-viewer>
- GLB models
- Mobile browser AR

Demo utilities:

- QR code generator
 - Local storage or simple JSON files
 - No auth
 - No database
-

5-hour feasibility

Feasible, but only with a fallback.

The risky part is real-time 3D generation. Text/image-to-3D APIs exist, but generation time, quality, and API setup may be unpredictable during a hackathon. So we should build two modes:

Mode 1: High-fidelity mode

Sketch + text

- OpenAI vision prompt refinement
- Meshy generates custom 3D model
- AR preview

Mode 2: Safe demo mode

Sketch + text

- OpenAI vision classifies product
- choose closest placeholder model
- Codex generates AR page + build pack
- AR preview works reliably

Prepare fallback models:

/models/bottle.glb
/models/lamp.glb
/models/chair.glb
/models/box.glb
/models/device.glb
/models/shoe.glb

This keeps the demo alive even if 3D generation fails.

5-hour build plan

Hour 1 — App shell

- Next.js setup
- Upload image
- Text prompt input
- Basic result screen
- Mobile AR route

Hour 2 — Sketch analysis

- Call OpenAI vision
- Extract product type, features, material, use case
- Generate refined 3D prompt

Hour 3 — AR viewer

- Add <model-viewer>
- Load .glb model
- Add QR code to mobile AR page
- Test on phone

Hour 4 — 3D generation + fallback

- Integrate Meshy API if keys/docs are ready
- Add fallback model selector
- Save product config JSON

Hour 5 — Codex angle + polish

- Generate AGENTS.md
 - Generate MVP_SPEC.md
 - Generate validation plan
 - Show “Codex generated these files” screen
 - Record 2-minute demo
-

What we should demo

Use a simple product shape:

Smart water bottle
Desk lamp
Small desk gadget
Packaging box
Chair

Best demo example:

Input image:

rough sketch of a bottle

Text:

“A smart water bottle for gym users that glows when hydration is low.”

Output:

- AI understands it as a smart hydration bottle
 - 3D model generated or fallback bottle selected
 - AR page generated
 - QR code scanned on phone
 - bottle placed on table in AR
 - Codex Build Pack generated
-

Biggest risks

Risk 1: 3D generation takes too long or looks bad

Mitigation: fallback model library

Risk 2: AR does not work on judge’s phone

Mitigation: test on our own Android/iPhone early, show screen recording backup

Risk 3: Judges think it is just Meshy + AR viewer

Mitigation: emphasize Codex-generated AR app, AGENTS.md, product config, and build pack

Risk 4: Too much scope

Mitigation: no auth, no database, no native mobile, no multi-user support

Final feasibility verdict

Buildable in 5 hours: Yes, with fallback models

Winning potential: High if demo works on phone

Main dependency risk: 3D generation API

Best positioning: Codex-powered sketch-to-AR prototype generator

Final pitch:

Reality MVP turns a sketch and product description into a spatial prototype. OpenAI understands the idea, a 3D API creates the model, and Codex generates the runnable AR app and build pack so the product can be seen in the real world immediately.